

```

##
=====
#####
## L2 Irish Lexical Attrition Study – Analysis Script
## Implements the analysis plan in codebook v22, Section F.
##
=====
#####
##
## HOW TO USE THIS SCRIPT
## Fill in the CONFIGURATION section below once real data files
exist, then
## run the whole script top to bottom. Sections marked "ADAPT"
depend on the
## exact column names your platform export uses – these can't be
finalised
## until the real export exists, so check them against your actual
file
## before trusting the output. Everything else should run as-is.
##
## REQUIRED PACKAGES (install once): lme4, ordinal, dplyr, readr,
readxl, broom
##
=====
=====

library(dplyr)
library(readr)
library(readxl)
library(lme4)
library(ordinal) # for clmm() – the ordinal mixed-effects model
library(broom)
library(broom.mixed)

##
-----
#####
## CONFIGURATION – fill these in once real files exist
##
-----

task_export_path      <- "task_trial_data.csv"          # long
format, one row per trial (see task instructions doc)
questionnaire_export_path <- "questionnaire_data.csv"    # one row
per participant
wordlist_path          <- "irish-attrition-wordlist-template-
v13.xlsx"

##
-----
## 1. LOAD RAW DATA
##

```

```

-----
# Task data – expected columns per the task instructions doc's data
export spec:
# participant_id | item_id | phase (recall/recognition) |
  irish_target |
# english_gloss | category | response_text | response_time_ms |
  live_match_result
task_raw <- read_csv(task_export_path)

# Questionnaire data – one row per participant, one column per item
(Q1, Q2, ... Q52)
# ADAPT: exact column naming convention depends on your platform's
export format
# (e.g. "Q16" vs "q16" vs "item_16" vs the item's internal field
name). Rename as
# needed so the column-selection code below finds the right columns.
quest_raw <- read_csv(questionnaire_export_path)

# Word list – for item-level word length and frequency rank
covariates
wordlist <- read_excel(wordlist_path, sheet = "FINAL 60 – Selected")

##
-----

## 2. SCORE THE LEXICAL TASK – build the three-level outcome per
item/participant
##
-----

## Per codebook Section E: this script assumes FINAL scoring (fada-
tolerant,
## edit-distance-tolerant, manual-review-resolved) has already been
applied to
## task_raw$live_match_result, i.e. this column already reflects
final correct/
## incorrect judgments, not just the platform's live routing check.
If your
## export still has raw typed responses only, do the scoring pass
(Section E)
## before running this script.

outcome_data <- task_raw %>%
  group_by(participant_id, item_id) %>%
  summarise(
    recalled = any(phase == "recall" & live_match_result == TRUE),
    recognized = any(phase == "recognition" & live_match_result ==
TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    # 0 = neither, 1 = recognized only, 2 = recalled – matches

```

```

codebook Section B
  retention_level = case_when(
    recalled ~ 2L,
    recognized ~ 1L,
    TRUE ~ 0L
  ),
  retention_level = ordered(retention_level, levels = c(0, 1, 2),
                           labels = c("neither",
"recognized_only", "recalled"))
)

##
-----

## 3. ITEM-LEVEL COVARIATES – word length and frequency rank
(codebook Section I)
##
-----

item_covariates <- wordlist %>%
  transmute(
    item_id = ID,
    category = Category,
    word_length = nchar(Irish),
    freq_rank = `Freq Rank`
  )

outcome_data <- outcome_data %>%
  left_join(item_covariates, by = "item_id")

##
-----

## 4. PARTICIPANT-LEVEL PREDICTORS – the 4 core predictors (codebook
Section C)
##
-----

## ADAPT: replace Qxx below with your actual exported column names
if different.

# --- Predictor 1: Proficiency at end of instruction
-----
# CA0-points-by-year conversion. This table is NOT filled in here –
the exact
# points-per-grade table differs by examination year and must be
sourced from
# the official CA0 points tables for each relevant year in your
sample. Build
# a lookup (year, level, grade) -> points, then join it in below.
#
# cao_lookup <- read_csv("cao_points_lookup.csv") # columns:
exam_year, level, grade, points

```

```

#
# quest_raw <- quest_raw %>%
#   left_join(cao_lookup, by = c("Q12" = "exam_year", "Q13" =
# "level", "Q14" = "grade"))
#
# For now, proficiency is left as a placeholder – replace with the
# joined
# `points` column once the CAO lookup is built.
quest_raw$proficiency <- NA_real_ # TODO: replace with CAO-
converted points

# --- Predictor 2: Exposure since end of formal instruction (Q16–
Q20) -----
exposure_items <- c("Q16", "Q17", "Q18", "Q19", "Q20")
quest_raw$exposure <- rowMeans(quest_raw[, exposure_items], na.rm =
TRUE)

# --- Predictor 3: Integrative/cultural motivation (Q21–Q24)
-----
integrative_items <- c("Q21", "Q22", "Q23", "Q24")
quest_raw$integrative_motivation <- rowMeans(quest_raw[,
integrative_items], na.rm = TRUE)

# --- Predictor 4: Multilingualism – count of actively-used
additional languages
# ADAPT: this depends heavily on how your platform exports Part 5's
repeating
# language block (first 2 languages, full detail) plus the overflow
list (3rd+,
# name + frequency only). The logic below assumes wide-format
columns
# lang1_freq, lang2_freq for the first two, and a semicolon-
separated overflow
# text field "lang_overflow" like "French: Occasionally; Spanish:
Rarely".
# Rewrite this block to match your actual export structure.

is_active <- function(freq) freq %in% c("Occasionally", "Often",
"Daily or almost daily")

count_overflow_active <- function(overflow_text) {
  if (is.na(overflow_text) || overflow_text == "") return(0)
  entries <- strsplit(overflow_text, ";")[[1]]
  sum(sapply(entries, function(e) any(sapply(c("Occasionally",
"Often", "Daily"), grepl, e))))
}

quest_raw$multilingualism <-
  is_active(quest_raw$lang1_freq) +
  is_active(quest_raw$lang2_freq) +
  sapply(quest_raw$lang_overflow, count_overflow_active)

# --- Secondary/exploratory predictor: Instrumental motivation (Q25–
Q26) -----

```

```

instrumental_items <- c("Q25", "Q26")
quest_raw$instrumental_motivation <- rowMeans(quest_raw[,
instrumental_items], na.rm = TRUE)

##
-----
## 5. MERGE – combine trial-level task data with participant-level
predictors
##
-----

model_data <- outcome_data %>%
  left_join(quest_raw, by = "participant_id")

cat("Merged dataset:", nrow(model_data), "rows (expect ~",
    length(unique(model_data$participant_id)) * 60, "for n
participants x 60 items)\n")

##
-----
## 6. PRIMARY ANALYSIS – trial-level mixed-effects ordinal model
##
-----

## Codebook Section F: by-participant and by-item random effects, 4
core
## predictors as fixed effects, plus word length and frequency rank
as
## item-level covariates.

primary_model <- clmm(
  retention_level ~ proficiency + exposure + integrative_motivation
+ multilingualism +
  word_length + freq_rank +
  (1 | participant_id) + (1 | item_id),
  data = model_data
)

cat("\n==== PRIMARY MODEL (confirmatory, 4 predictors) =====\n")
summary(primary_model)

##
-----
## 7. SECONDARY ANALYSIS – exploratory 5-predictor model
##
-----

## Adds instrumental motivation. Codebook flags this as underpowered
## (n=40, 5 predictors = 8 obs/predictor, below the 10/predictor
guideline) –

```

```
## report with wider confidence intervals and interpret cautiously,  
not as  
## a confirmatory result.
```

```
secondary_model <- clmm(  
  retention_level ~ proficiency + exposure + integrative_motivation  
+ multilingualism +  
  instrumental_motivation +  
  word_length + freq_rank +  
  (1 | participant_id) + (1 | item_id),  
  data = model_data  
)
```

```
cat("\n===== SECONDARY MODEL (exploratory, 5 predictors) =====\n")  
cat("NOTE: underpowered relative to primary model – interpret  
cautiously.\n")  
summary(secondary_model)
```

```
##
```

```
-----
```

```
## 8. TERTIARY ANALYSIS – simpler participant-level check  
##
```

```
-----
```

```
## Codebook Section F: "a simpler participant-level regression,  
using averaged  
## composite scores, is reported as a supplementary check on the  
primary  
## model's conclusions."
```

```
participant_summary <- model_data %>%  
  group_by(participant_id) %>%  
  summarise(  
    mean_retention = mean(as.numeric(retention_level) - 1), #  
    rescale 0/1/2  
    proficiency = first(proficiency),  
    exposure = first(exposure),  
    integrative_motivation = first(integrative_motivation),  
    multilingualism = first(multilingualism)  
  )
```

```
tertiary_model <- lm(  
  mean_retention ~ proficiency + exposure + integrative_motivation +  
  multilingualism,  
  data = participant_summary  
)
```

```
cat("\n===== TERTIARY MODEL (participant-level check) =====\n")  
summary(tertiary_model)
```

```
##
```

```
-----
```

```

## 9. RELIABILITY CHECKS – Cronbach's alpha for each composite
(codebook F)
##
-----
-----
## Requires the 'psych' package: install.packages("psych")

library(psych)

cat("\n===== COMPOSITE RELIABILITY (Cronbach's alpha) =====\n")
cat("Exposure (Q16–20):\n"); print(psych::alpha(quest_raw[,
exposure_items])$total$raw_alpha)
cat("Integrative motivation (Q21–24):\n");
print(psych::alpha(quest_raw[, integrative_items])$total$raw_alpha)
cat("Instrumental motivation (Q25–26) – 2 items, report correlation
not alpha:\n")
print(cor(quest_raw$Q25, quest_raw$Q26, use = "complete.obs"))

##
-----
-----
## 10. COLLINEARITY CHECK – VIF on the primary model's predictors
##
-----
-----
## Codebook F flags this explicitly, given the known correlation
between
## proficiency and motivation in Geary (2022).

library(car)
participant_level_check <- lm(
  mean_retention ~ proficiency + exposure + integrative_motivation +
  multilingualism,
  data = participant_summary
)
cat("\n===== VIF CHECK =====\n")
print(vif(participant_level_check))

##
=====
=====
## END OF SCRIPT
##
=====
=====

```